学校代码：11068

学号：202052320220

密级：内部

郑州工程技術学院

（ 2024 ）届毕业设计

基于深度学习的智能车辆识别系统设计与实现

作者姓名：王盼

导师姓名（职称）：王国政（讲师）

学科门类：工学

专业名称：智能科学与技术

培养学院：信息工程学院

完成时间：2024 年 5 月

郑州工程技术学院
2024 届毕业设计任务书

<table>
<tr><td colspan="2">毕业设计题目</td><td colspan="6">基于深度学习的智能车辆识别系统设计与实现</td></tr>
<tr><td>学　院</td><td colspan="3">信息工程学院</td><td>专　业</td><td colspan="3">智能科学与技术</td></tr>
<tr><td>学生姓名</td><td>王盼</td><td>学号</td><td>202052320220</td><td>指导教师</td><td>王国政</td><td>职称</td><td>讲师</td></tr>
<tr><td>任务规定进行日期</td><td colspan="7">自 2023 年 11 月起，至 2024 年 5 月止</td></tr>
</table>

一、　毕业设计的目的、要求

目的：旨在通过基于深度学习的车辆识别检测系统，将理论知识转化为实践能力，培养应用技术和解决问题的能力。此项目对深化专业知识和未来职业生涯或研究具有重要意义，期望展现出创新和操作能力。

要求：毕业设计论文撰写时，应确保文献资料全面、系统、科学，表现研究的独立性和深度。论文表述须清晰、严谨、流畅，反映出较高的写作能力和组织能力。系统设计需遵守行业规范，展示系统效果，包括车辆识别准确率和交通管理控制效果，以评估项目成效。

二、　主要工作内容

阶段一：选择匹配车辆识别检测系统的算法，收集和预处理车辆数据。

阶段二：训练车辆识别模型，进行初步测试和参数调优。

阶段三：在多样化交通环境中进行模型测试，确保准确性和稳定性。

阶段四：将模型集成至智能交通系统，进行用户界面优化。

阶段五：撰写毕业论文，总结项目并规划未来研究方向。

三、　主要设计指标

（1）算法选择：挑选符合车辆识别系统的深度学习算法，适应多样交通场景。

（2）模型训练：采用交叉验证法优化模型参数，注重训练数据的质量。

（3）泛化测试：确保模型在不同的光照、天气和车辆类型下保持性能。

（4）准确率评估：测试模型以保证高准确率的车辆识别。

（5）系统稳定性：强化系统以适应交通环境变化，减少故障。

（6）系统可靠性：确保系统在多变条件下的安全性和有效运行。

四、 进度计划

2023.11.01~2023.12.15：启动项目，进行深度学习及车辆识别系统的调研，提交任务书确定研究方向。

2023.12.16~2024.1.15：研究深度学习算法，挑选适合车辆识别的算法，开始数据收集与预处理，撰写开题报告。

2024.1.16~2024.2.28：训练车辆识别模型，进行初步测试和参数调优。

2024.3.1~2024.3.31：对模型进行进一步训练和优化，测试准确率和泛化能力，分析测试结果。

2024.4.1~2024.4.30：完成模型优化，将模型集成到交通监控系统中测试，调整系统性能，撰写毕业论文。

2024.5.1~2024.5.12：完成毕业论文撰写，准备答辩，确保清晰展示研究成果。

五、主要参考资料（外文资料至少一篇）

[1] 杨雄.基于视频的车辆多属性识别系统的设计与实现[D].内蒙古大学,2023.DOI：10.27224/d.cnki.gnmdu.2023.000972.

[2] 王利群.基于深度学习的车辆多属性识别方法研究[D].江苏科技大学,2022.DOI：10.27171/d.cnki.ghdcc.2022.000445.

[3] 胡佳睿.基于边缘平台的车辆智能分析系统[D].华中科技大学,2020.DOI：10.27157/d.cnki.ghzku.2020.006250.

[4] 张子昊.基于视频的实时车辆检测与属性识别算法研究及系统实现[D].北京工业大学,2020.DOI：10.26935/d.cnki.gbjgu.2020.001078.

[5] Novak, Leslie M., et al. "The automatic target-recognition system in SAIP." Lincoln Laboratory Journal 10.2 (1997).

学生签字	日期： 年 月 日	指导教师签字	日期： 年 月 日

教研室审批意见：

教研室：

院领导审核意见：

院领导：

郑州工程技术学院
学位论文原创性声明

本人郑重声明：所呈交的学位论文是本人在导师的指导下独立进行研究所取得的研究成果。除文中已经加以标注引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

作者签名：　　日期:　　年　月　日

郑州工程技术学院
学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权郑州工程技术学院可以将本学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

本学位论文属于

1、保密□，在____年解密后适用本授权书。

2、不保密□。

（请在以上相应方框内打“√”）

作者签名：　　日期:　　年　月　日

导师签名：　　日期:　　年　月　日

目 录

摘 要 1
Abstract 2
1 绪论 3
1.1 研究背景与意义 3
1.2 国内外研究现状 3
1.3 论文的主要工作与结构安排 4
2 相关技术背景 5
2.1 计算机视觉 5
2.2 深度学习 6
2.2.1 卷积神经网络（CNN） 6
2.2.2 目标检测算法（YOLO） 7
2.2.3 目标跟踪算法（ByteTrack） 7
3 智能车辆识别系统设计 8
3.1 系统总体设计 8
3.1.1 系统架构 8
3.1.2 主界面设计 9
3.2 车辆属性识别模块 10
3.2.1 车辆属性识别流程 10
3.2.2 车辆属性识别算法设计 12
3.2.3 车辆属性识别界面设计 13
3.3 车辆速度检测模块 14
3.3.1 速度检测流程 14
3.3.2 速度计算方法 16
3.3.3 车速检测界面设计 17
3.4 车流量统计模块 18
3.4.1 车流量统计流程 18

3.4.2 车辆计数方法 19
3.4.3 车流量统计界面设计 20
3.5 车辆停留时间检测模块 21
3.5.1 停留时间检测流程 22
3.5.2 停留时间计算方法 23
3.5.3 车辆停留时间分析界面设计 24
4 边缘设备移植与优化加速 24
4.1 C++代码移植 25
4.1.1 C++版本的系统架构 25
4.1.2 关键模块的 C++实现 26
4.2 优化加速视频 FPS 29
4.2.1 模型导出与量化 29
4.2.2 线程池优化 29
4.3 移植效果性能评估 29
5 总结与展望 31
5.1 工作总结 31
5.2 未来展望 32
致谢 33
参考文献 34

摘 要

随着城市道路交通压力日益增大，传统交通管理方式面临越来越多的挑战。然而，在软硬件资源有限的情况下，当前的智能交通管理系统在车辆识别方面仍存在识别精度不高、处理速度较慢等问题，难以满足对实时、高精度车辆识别的需求，在一定程度上影响了交通管理效率和出行体验。本设计针对上述问题，在有限的软硬件资源下，设计并实现了一种基于深度学习的智能车辆识别系统。

本系统综合利用计算机视觉和深度学习算法，首先，在车辆属性识别方面，采用卷积神经网络实现对车牌号码、车牌颜色和车身颜色的精确识别。其次，车速检测通过对连续帧图像进行透视变换和目标跟踪，结合位移计算车速。车流量统计则利用目标检测和跟踪算法，结合虚拟线圈和方向判断，实现车流量的实时统计。最后，停留时间检测引入了目标区域，通过跟踪车辆持续计算并显示车辆在区域内的停留时间，为停车管理和违章检测提供支持。

本系统基于PyQt5还设计了友好的图形用户界面，用户可以方便地配置参数、选择功能模块。此外，本系统还针对边缘设备平台进行了优化和加速。经评估，本系统在车辆属性识别、车速检测、车流量统计、停留时间检测等多项功能上实现了较好的性能表现和处理效率。

关键词：智能交通；车辆识别；深度学习；目标检测；目标跟踪

Design and implementation of intelligent vehicle identification system based on deep learning

Abstract

As urban road traffic continues to intensify, traditional traffic management methods increasingly face significant challenges. In the context of limited hardware and software resources, current intelligent traffic management systems still exhibit shortcomings, such as low accuracy in vehicle identification and slow processing speeds, which fail to meet the requirements for real-time, high-precision vehicle recognition and adversely affect traffic management efficiency and travel experience. To address these issues, a smart vehicle recognition system based on deep learning has been designed and implemented with limited hardware and software resources.

This system leverages computer vision and deep learning algorithms comprehensively. Initially, convolutional neural networks (CNNs) are utilized to precisely identify vehicle attributes, including license plate numbers, license plate colors, and vehicle body colors. Subsequently, vehicle speed detection is conducted by performing perspective transformations and target tracking on consecutive frame images, combined with displacement calculations to determine speed. Traffic flow statistics are generated using object detection and tracking algorithms, alongside virtual loop detectors and directional analysis for real-time monitoring. Furthermore, dwelling time detection has been enhanced by introducing a targeted area of interest, continually tracking and displaying the time vehicles spend in designated areas, thus supporting parking management and violation detection.

The system also features a user-friendly graphical user interface (GUI) developed with PyQt5, allowing users to easily configure settings and select functional modules. Moreover, the system has been optimized and accelerated for edge computing platforms. Evaluations demonstrate that this system achieves robust performance and processing efficiency across multiple functions, including vehicle attribute recognition, speed detection, traffic flow analysis, and dwelling time monitoring.

Keywords： Intelligent Transportation; Vehicle Recognition; Deep Learning; Target Detection; Target Tracking

基于深度学习的智能车辆识别系统设计与实现

王盼 202052320220

1 绪论

1.1 研究背景与意义

随着城市化进程的不断推进和汽车保有量的持续增长，交通管理面临着越来越大的挑战。传统的交通管理方式已难以满足日益增长的交通需求，需要引入智能化技术手段来提升交通管理的效率和精细化水平。

智能车辆识别系统作为一种将先进的信息技术、通信技术、传感技术、控制技术等应用于交通管理的系统，已成为缓解交通拥堵、提高交通安全、优化交通资源配置的重要手段。而车辆识别技术作为智能交通系统的核心组成部分，在交通监控、交通流量统计、违章检测等方面发挥着关键作用。

传统的车辆识别方法主要依赖于地感线圈、雷达、红外等传感器，存在成本高、维护困难、容易受环境影响等问题。近年来，随着计算机视觉和深度学习技术的飞速发展，利用视频图像进行车辆识别成为了一种更加灵活、经济、智能的解决方案。基于深度学习的车辆识别方法可以从图像或视频中自动提取车辆的特征信息，实现车辆属性（车牌号码、颜色，类型和车辆颜色等）的识别，车辆速度的估计，车流量的统计以及车辆停留时间检测等多项功能。

本系统旨在设计和实现一个基于深度学习的智能车辆识别系统，该系统综合利用计算机视觉、深度学习和图形用户界面设计等技术，提供车辆属性识别、车辆速度检测、车流量统计和车辆停留时间检测等功能，为智能车辆管理提供有力的技术支撑。这不仅可以提高交通管理的智能化水平，还可以为交通规划、交通控制和交通执法等提供数据支持和决策依据，具有重要的研究意义和应用价值。

1.2 国内外研究现状

国外研究现状：国外在智能车辆识别领域的研究起步较早，理论和技术较为成熟。近年来，国外研究者将深度学习引入车辆识别，极大提升了识别性能。在车辆检测方面，国外团队提出了YOLO、FasterR-CNN等经典模型，实现了车辆的实时、精确定位。在车牌识别方面，国外研究者Li，Hui[1]等人提出了基于CNN和RNN的端到端识别模型，将车牌检测和字符识别统一到一个网络框架中，大幅提高了识别效

率和准确性。同时，针对车牌图像数量不足问题，国外研究者Shvai，Nadiya[2]等人提出了基于GAN的数据增强方法，通过生成逼真的车牌图像样本，扩充了训练数据集。在车辆跟踪和速度估计方面，国外研究者Wang，Zhangjing[3]等人充分利用视觉、雷达、激光雷达等多源传感器信息，设计了多传感器融合的车辆跟踪和速度估计系统。其中，DeepSORT算法引入深度特征表示，结合卡尔曼滤波，实现了车辆的长时跟踪。总的来看，国外在智能车辆识别领域积累了丰富的研究成果，代表了该领域的最新进展。

国内研究现状：与国外相比，国内在智能车辆识别领域的研究起步较晚，但近年来呈现出追赶态势，涌现出一批有影响力的研究成果。在车辆检测和识别方面，张健[4]提出了多种改进的检测模型，如基于注意力机制的车辆检测网络、针对遮挡问题的分段式检测方法等，在复杂交通场景下展现了良好的性能。针对车牌识别，国内研究者针对性地设计了符合国情的车牌识别模型，于淏[5]提出了基于注意力机制的车牌识别算法，有效提升了车牌字符的识别率。车辆跟踪和速度估计方面，曾伟良，聂佩林，何兆成[6]提出了结合神经网络和数据融合技术的城市路段速度估计，设计了鲁棒、实时的跟踪和测速系统，为智能交通管控提供了有力工具。总的来看，国内在智能车辆识别领域的研究正蓬勃发展，部分研究已达到国际先进水平。

综上所述，智能车辆识别是计算机视觉与智慧交通的前沿交叉方向，对于提升交通安全、提高通行效率具有重要意义。国外研究经过数十年积累，已形成系统、全面的理论和技术体系，代表了当前智能车辆识别的最高水平。相比之下，国内虽起步较晚，但近年来发展迅速，在车辆检测、车牌识别等方面取得了显著进展，正逐步缩小与国外的差距。

1.3 论文的主要工作与结构安排

本系统的主要工作是设计和实现一个基于深度学习的智能车辆识别系统。具体包括以下几个方面：

（1）车辆属性识别：针对车牌识别和车辆颜色识别任务，利用卷积神经网络构建识别模型，实现车牌号码、车牌颜色、车牌类型和车身颜色的精确识别。

（2）车辆速度检测：使用一种基于透视变换和目标跟踪的车辆速度估计方法。通过将检测到的车辆映射到鸟瞰视角，并计算车辆在连续帧中的位移，实现车速估计。

（3）车流量统计：利用YOLO算法进行车辆检测，采用ByteTrack算法进行车辆

跟踪，通过虚拟线圈和方向判断实现车流量的统计。

（4）车辆停留时间检测：通过在视频中定义多边形区域，并结合目标检测和跟踪技术，实现对指定区域内车辆停留时间的计算。

（5）图形用户界面设计：采用PyQt5框架，设计了一套友好的图形用户界面，实现图片或视频上传、摄像头连接、车辆分析结果显示等交互功能。

全文共分为四章，具体结构如下：

第一章为绪论，介绍了研究背景与意义、国内外研究现状以及论文的主要工作与结构安排。

第二章为相关技术背景，介绍了计算机视觉、深度学习等相关技术基础。

第三章为智能车辆识别系统设计，详细阐述了系统的总体设计、各功能模块的设计思路和实现方法，以及图形用户界面的设计。

第四章为边缘设备移植与优化加速，介绍了将算法移植到C++并在边缘平台上进行优化加速的过程，包括C++代码实现、模型量化、线程池优化等内容，并对移植效果与性能进行了评估。

第五章为总结与展望，对全文工作进行了总结，并对未来的研究方向进行了展望。

2 相关技术背景

本章将介绍智能车辆识别系统涉及的相关技术背景，主要包括计算机视觉、深度学习等方面的内容。

2.1 计算机视觉

计算机视觉是智能车辆识别系统的重要基础，其目标是使计算机能够从图像或视频中获取有意义的信息，并进行理解和分析。计算机视觉包括图像处理和视频处理两个主要方面。

图像处理是将图像转换为更适合人类观察或机器分析的形式。常见的图像处理操作包括灰度化、二值化、图像增强和图像变换等。灰度化是将彩色图像转换为灰度图像，常用的方法有分量法、最大值法和加权平均法[7]。二值化是将灰度图像转换为只包含黑白两种像素值的图像，常用的方法有全局阈值法和自适应阈值法[8]。图像增强旨在提高图像的视觉质量，突出图像的某些特征，如直方图均衡化、锐化和平滑等。直方图均衡化通过调整图像的灰度分布，使其更加均匀，增强图像的对比度。

视频处理可以看作是对图像序列的处理，常见的操作包括视频帧的提取与保存、背景建模与前景检测、光流法和目标跟踪等。视频帧的提取与保存是视频处理的基础，通过从视频中读取帧图像并将处理后的帧图像保存为视频，可以实现对视频的分析和处理。背景建模与前景检测通过建立视频的背景模型，并将当前帧与背景模型比较，提取出运动的前景目标。光流法通过估计相邻帧之间像素的运动情况[9]，可以用于运动目标检测和跟踪。目标跟踪则是在视频中连续定位感兴趣的目标，是许多视频分析任务的重要组成部分[10]。

在智能车辆识别系统中，计算机视觉技术发挥着关键作用。通过对图像和视频的处理与分析，可以实现车辆检测、属性提取、速度估计等功能。例如，利用目标检测算法可以从图像或视频帧中定位车辆的位置；通过图像分割和特征提取，可以识别车辆的颜色、车牌号码等属性；结合目标跟踪和几何计算，可以估计车辆的速度和轨迹。因此，深入理解和应用计算机视觉技术，对于开发高效、准确的智能车辆识别系统具有重要意义。

2.2 深度学习

深度学习是机器学习的一个分支，其目标是通过构建具有多层结构的人工神经网络，使计算机能够从数据中学习到高层次的抽象特征[11]。深度学习在计算机视觉、语音识别、自然语言处理等领域取得了重大突破，极大地推动了人工智能的发展。

2.2.1 卷积神经网络（CNN）

卷积神经网络（Convolutional Neural Network， CNN）是一种专门用于处理网格拓扑结构数据（如图像）的深度神经网络[12]。CNN通过卷积层和池化层的组合，能够自动学习到图像的层次化特征表示，在图像分类、目标检测等任务上表现出色。

CNN的基本组成部分包括：

（1）卷积层：通过卷积操作提取局部特征。

（2）池化层：通过下采样操作减小特征图的尺寸，提高特征的鲁棒性。常用的池化方式有最大池化和平均池化。

（3）全连接层：将卷积层和池化层提取的特征映射到样本的类别标签上。

通过这些层的串联，CNN能够逐层提取图像从低级到高级的特征，并用于分类或回归任务。常用的CNN架构包括LeNet、AlexNet、VGGNet、GoogLeNet等。

2.2.2 目标检测算法（YOLO）

目标检测是计算机视觉的一项基本任务，旨在从图像中定位和识别出感兴趣的目标。YOLO（You Only Look Once）是一种高效的实时目标检测算法，它将目标检测问题转化为一个回归问题，通过单次前向传播即可直接预测出目标的边界框和类别概率[13]。

YOLO将输入图像划分为$S \times S$个网格，每个网格负责检测中心落在该网格内的目标。对于每个网格，YOLO预测B个边界框，以及这些边界框所属的C个类别的概率。因此，YOLO的输出张量维度为$S \times S \times (B * 5 + C)$，其中5表示每个边界框的5个参数（中心坐标、宽、高和置信度）。

YOLO通过端到端的方式训练整个网络，可以实现实时的目标检测[14]。YOLO算法的改进版本，如YOLOv2、YOLOv3、YOLOv4、YOLOv5等，进一步提高了检测精度和速度。

2.2.3 目标跟踪算法（ByteTrack）

目标跟踪旨在从视频序列中持续定位感兴趣的目标，是智能视频分析的一项关键技术。ByteTrack是一种高效的多目标跟踪算法，通过将检测结果与跟踪轨迹进行关联，实现了对目标的持续跟踪[15]。

ByteTrack的主要思路是：

（1）将检测器得到的目标边界框与现有的跟踪轨迹进行匹配，计算它们之间的相似度。相似度基于边界框的IoU（Intersection over Union）、外观特征的余弦距离等。

（2）对于没有匹配到跟踪轨迹的检测结果，创建新的跟踪轨迹。

（3）对于没有匹配到检测结果的跟踪轨迹，如果其连续未匹配的次数超过一定阈值，则将其删除。

（4）利用卡尔曼滤波等算法，基于匹配到的检测结果更新跟踪轨迹的状态[16]（如位置、速度等）。

ByteTrack引入了一种简单而有效的跟踪策略，即将跟踪问题分解为两个子问题：将检测结果与跟踪轨迹关联，以及恢复被遮挡或暂时丢失的目标。通过这种方式，ByteTrack可以处理目标的出现、消失和相互遮挡等复杂情况，实现稳定可靠的多目标跟踪。

深度学习技术，特别是CNN、YOLO和ByteTrack算法，为智能车辆识别系统的

开发提供了强大的工具。通过将这些算法应用于车辆检测、属性识别和跟踪等任务，可以实现对车辆的精确定位、识别和状态估计，为智能车辆管理提供数据支撑。

3 智能车辆识别系统设计

3.1 系统总体设计

3.1.1 系统架构

智能车辆识别系统采用模块化的设计思路，将复杂的系统功能划分为多个相对独立的模块。系统的总体架构如图3.1所示。

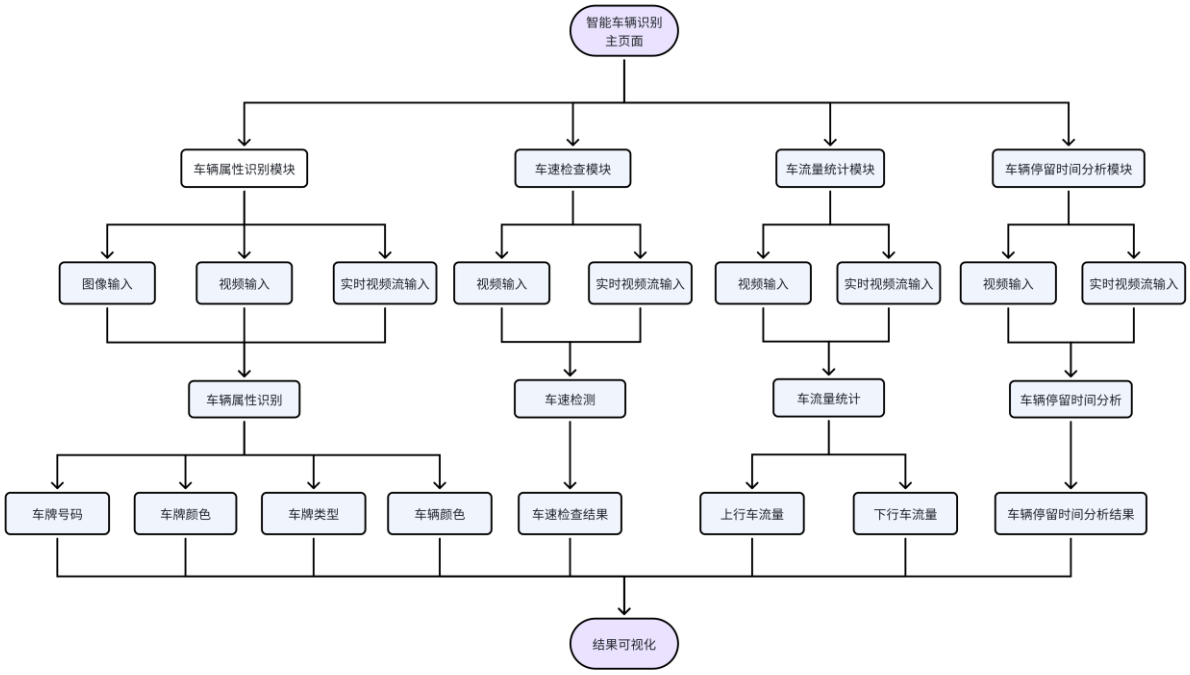

图3.1 智能车辆识别系统架构图

系统主要由以下几个部分组成：

（1）智能车辆系统主界面作为系统的入口，提供了车辆属性识别、车速检测、车流量统计和车辆停留时间分析四个主要功能模块的访问入口，使用PyQt5库开发图形用户界面，实现了友好的人机交互。

（2）车辆属性识别模块接受图像、视频和实时视频流三种类型的输入数据，使用YOLO算法对输入数据进行车辆检测、定位车辆位置。对检测到的车辆进行属性识别，包括车牌识别和车身颜色识别两个子任务：车牌识别通过图像预处理、字符分割和字符识别等步骤识别出车牌号码，其中字符识别使用卷积神经网络实现；车身颜色识别对车身区域提取颜色特征，并使用支持向量机或神经网络等机器学习算法

进行颜色分类。将车辆属性识别的结果车牌号码、车牌颜色、车牌类型和车身颜色传递给数据可视化模块进行展示。

（3）车速检测模块接受视频和实时视频流两种类型的输入数据，使用YOLO算法对视频帧进行车辆检测，并使用ByteTrack算法实现车辆跟踪、获取车辆的位置信息，通过透视变换将车辆位置从图像坐标系转换到鸟瞰图坐标系中消除透视效应，根据车辆位置的变化和时间间隔，计算车辆的瞬时速度和平均速度，速度检测的结果将传递给数据可视化模块进行展示。

（4）车流量统计模块接受视频和实时视频流两种类型的输入数据，使用YOLO算法对视频帧进行车辆检测、获取车辆位置，在图像中设置虚拟线圈，通过判断车辆通过虚拟线圈的方向，对上行和下行车辆分别进行计数，将车流量统计的上行车流量和下行车流量结果传递给数据可视化模块进行展示。

（5）车辆停留时间分析模块接受视频和实时视频流两种类型的输入数据，使用YOLO算法对视频帧进行车辆检测、获取车辆位置，在图像中设置目标区域，通过跟踪车辆在ROI内的出现时间和离开时间，计算车辆的停留时间。

（6）数据可视化模块汇聚车辆属性识别、车速检测、车流量统计和车辆停留时间分析模块的输出结果，通过PyQt5开发的用户界面，将可视化结果呈现给用户，从而实现人机交互。

这样的架构图清晰地展示了智能车辆识别系统的模块组成和数据流向，突出了车辆检测与跟踪作为核心模块，各个功能模块并行处理的特点，以及用户交互界面在系统中的重要作用，帮助读者全面理解系统的整体架构和工作原理。同时，也体现了系统的可扩展性，新的功能模块可以方便地接入到车辆检测与跟踪模块的输出，实现系统功能的不断丰富和优化。

3.1.2 主界面设计

系统的主界面是用户进入系统后的第一个页面，需要提供清晰、直观的功能导航和系统介绍，如图3.2所示。主界面的设计要点如下：

（1）系统标题：在界面顶部显示系统的名称“智能车辆识别系统”，突出系统的功能特色。

（2）功能按钮：提供四个大的功能按钮，分别对应系统的四个主要模块：车辆属性识别、速度检测、车流量检测和车辆停留时间检测。按钮使用扁平化设计风格，配以醒目的图标和文字说明。

（3）系统说明文字：在界面中部提供一段简短的系统说明文字，概括系统的功能和特点，吸引用户的兴趣。

图3.2 系统主页面

（4）顶部导航栏：在界面顶部设置导航栏，提供“帮助”链接，方便用户获取系统的更多信息。

（5）主界面的配色以清新、友好为主，采用蓝色、彩色等让人感觉舒适、可靠的色调。界面布局简洁大方，体现出系统的专业性和易用性。

3.2 车辆属性识别模块

车辆属性识别是智能车辆识别系统的核心功能之一，主要包括车牌识别和车身颜色识别两个子任务。本节将详细介绍车辆属性识别模块的设计与实现。

3.2.1 车辆属性识别流程

车辆属性识别的整体流程如图3.3所示。

具体步骤如下：

（1）车辆检测模块

输入图像、视频、实时视频流数据，使用基于卷积神经网络的YOLO目标检测算法对输入数据进行车辆检测，一次性预测多个目标物体的边界框和类别概率，输出为检测到的车辆目标的边界框坐标[17]。

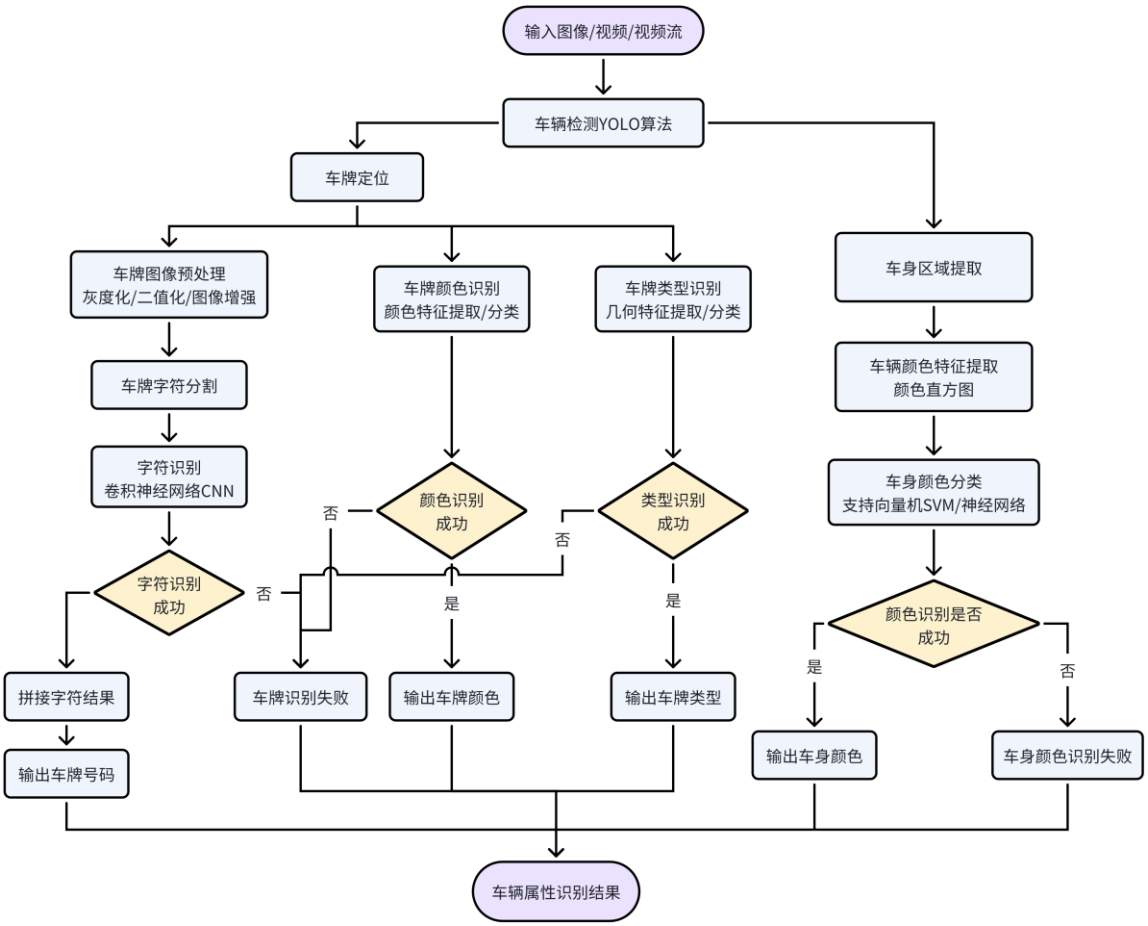

图3.3 车辆属性识别流程图

（2）车牌识别模块

利用车辆边界框，在车辆区域内通过边缘检测、颜色分割等方法定位车牌区域，得到车牌边界框。对定位的车牌图像进行灰度化、二值化、直方图均衡化等增强预处理。使用投影法或连通域分析将预处理的车牌图像分割为单个字符图像，再使用CNN对分割的单字符图像进行特征提取和分类，识别为字符结果并拼接为完整车牌号码。提取车牌颜色特征，使用分类器（SVM）识别车牌颜色。提取车牌几何特征，使用分类器识别车牌类型（单层车牌/双层车牌等）。

（3）车身颜色识别模块

根据车辆边界框和车牌位置，使用图像分割算法（语义分割）提取车身区域。对车身区域图像提取颜色特征。使用SVM等分类器，对提取的颜色特征向量进行车身颜色分类识别。

（4）结果输出

汇总各模块的识别结果，生成完整的车辆属性识别结果，包括车牌号码、车牌

颜色、车牌类型、车身颜色。若任一子模块识别失败，相应属性位置为空。

3.2.2 车辆属性识别算法设计

（1）车牌识别算法设计

车牌识别是车辆属性识别的关键任务，本系统采用基于卷积神经网络（CNN）端到端的识别方法。CNN具有强大的特征提取和分类能力，可以有效处理车牌图像的复杂性和多样性。

通过一个轻量级的CNN网络，用于同时实现车牌检测和字符识别。网络结构包括：输入层，接收大小为$W \times H \times 3$的车辆图像；卷积层，使用多个卷积层提取车牌图像的多尺度特征，卷积核大小为3×3，步长为1，激活函数为ReLU；池化层，使用最大池化层降低特征图的尺寸，池化核大小为2×2，步长为2；全连接层，将卷积层提取的特征展平，并使用全连接层进行特征融合和维度转换；输出层，使用两个并行的输出层，一个用于车牌检测（分类），一个用于字符识别（序列标注）。

网络的损失函数包括两部分：车牌检测损失和字符识别损失。车牌检测损失采用二元交叉熵损失，字符识别损失采用条件随机场（CTC）损失。总损失为两部分损失的加权和：

$$L = \lambda_1 L_{det} + \lambda_2 L_{rec} \tag{3.1}$$

其中，L_{det}和L_{rec}分别表示车牌检测损失和字符识别损失，λ_1和λ_2为平衡两部分损失的权重系数。

在推理阶段，输入车辆图像经过网络前向传播，得到车牌的位置和字符序列，实现车牌识别的目标。

（2）车辆颜色识别算法设计

车身颜色识别的目标是判断车辆的主要颜色，如红色、蓝色、白色等。采用基于颜色直方图的方法进行车身颜色识别，具体步骤如下：

① 车身区域提取：根据车辆检测的边界框，提取车身区域的图像。

② 颜色空间转换：将车身图像从RGB颜色空间转换到HSV颜色空间，以便更好地表示颜色信息。

③ 颜色直方图计算：对车身图像的H（色调）和S（饱和度）通道分别计算颜色直方图，得到颜色分布特征。

④ 直方图归一化：对计算得到的颜色直方图进行归一化处理，使得直方图的取值范围在[0，1]之间，便于后续的分类器训练和推理。

⑤ 颜色分类：使用预训练的颜色分类器（SVM），对提取的颜色直方图特征进行分类，得到车身颜色的类别标签。

车辆属性识别模块的设计，综合利用了计算机视觉和深度学习等技术，实现了车牌识别和车身颜色识别的功能。其中，车牌识别采用了端到端的CNN卷积神经网络，同时完成车牌检测和字符识别；车身颜色识别采用了颜色直方图和分类器的方法，有效提取车身的颜色特征[18]。

3.2.3 车辆属性识别界面设计

车辆属性识别界面分为图片模式和视频模式两个子界面，满足不同场景需求。

（1）图片模式界面

图3.4 图片识别页面

提供“上传图片”和“清除图片”按钮，加载/清空待识别图片。上方区域显示原始车辆图片，下方区域显示识别结果图片。下方文本框展示车牌号码、车牌颜色、车牌类型和车身颜色等属性信息。具体效果如图3.4所示。

（2） 视频模式界面

提供“上传视频”、“清除视频”、“启动摄像头”和“关闭摄像头”按钮。左侧显示原始视频画面，右侧同步显示识别结果。下方设有播放控制区域，包括“播放/暂停”、“重播”和“倍速”按钮。进度条和时间标签实时反映视频播放进度和当前时间。

底部设有“返回主界面”按钮，实现与系统主界面的无缝切换。为用户提供高效、友好的车辆属性识别操作环境。具体效果如图3.5所示。

图3.5 视频识别页面

3.3 车辆速度检测模块

车辆速度检测是智能车辆识别系统的一项重要功能，可以用于交通流量统计、超速监控等场景。本节将介绍车辆速度检测模块的设计与实现。

3.3.1 速度检测流程

车辆速度检测的整体流程如图3.6所示。

具体步骤如下：

（1）视频输入：系统支持两种视频输入方式，上传本地视频文件或使用实时摄像头。用户可根据需求选择合适的视频源。

（2）设置检测参数：在视频处理开始前，用户需要设置一些检测参数，包括选择检测区域坐标、使用的检测模型以及使用CPU或GPU设备。这些参数的设置对检测精度和效率有重要影响。

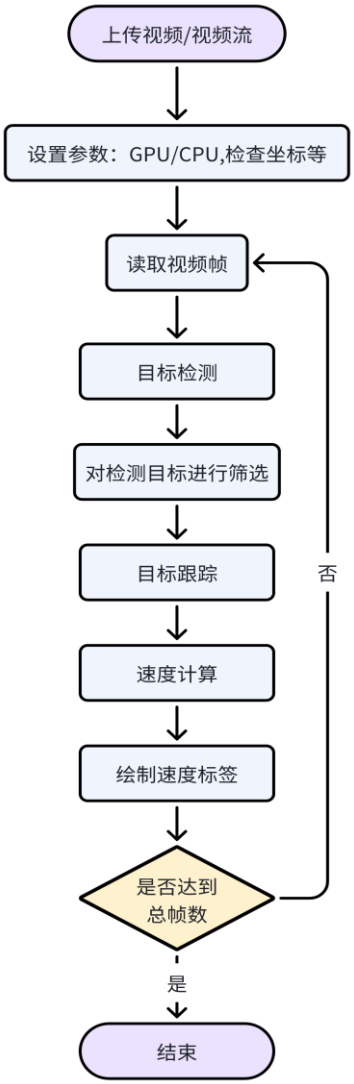

图3.6 车辆速度检测流程图

（3）视频处理：设置完参数后，系统开始对输入视频进行处理，采用逐帧处理方式，对每一帧执行以下步骤：

① 目标检测：使用YOLOv8模型对当前视频帧进行目标检测，识别出视频帧中的车辆

② 目标位置判断：判断检测到的车辆是否位于指定的检测区域内

③ 目标跟踪：对区域内的车辆进行跟踪，生成运动轨迹

④ 速度计算：根据车辆轨迹和时间信息，计算车辆的速度

⑤ 结果可视化：将检测结果、跟踪轨迹、速度等信息叠加到原始视频帧上

（4）输出结果：视频处理完成后，输出带有可视化检测结果的视频，同时显示进度条指示处理进度。

（5）用户交互控制：系统提供用户交互控制功能，用户可以通过控制命令对视频处理过程进行控制，如播放/暂停、重播、调整播放速度等。用户还可以选择返回主窗口。

3.3.2 速度计算方法

由于摄像机成像过程中不可避免地引入了透视效应，导致图像中车辆的位置与其在真实世界中的位置存在一定的偏差。为了准确估计车速，需要将车辆在图像坐标系下的位置转换到与真实世界对齐的坐标系中。本设计采用透视变换（Perspective Transformation）实现这一坐标映射过程。

透视变换是将图像从一个视平面转换到另一个视平面的几何变换[19]，常用于消除图像的透视失真。它可以建立图像坐标系与真实世界坐标系之间的映射关系。在车速估计任务中，需要将成像平面内的车辆位置映射到路面（地面）坐标系，以实现速度的准确计算。

假设路面为一个二维平面，定义路面坐标系为$O-XY$，其中O为坐标原点，X轴平行于车道方向，Y轴垂直于车道方向。图像坐标系定义为$o-uv$，其中o为图像左上角，u轴水平向右，v轴垂直向下。透视变换可以用一个3×3的矩阵$\boldsymbol{H}$来描述，将图像坐标(u,v)映射到路面坐标(x,y)：

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \boldsymbol{H} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \tag{3.2}$$

其中，$\boldsymbol{H}$称为单应性矩阵，可以通过已知的图像坐标和对应的真实世界坐标求解得到。在实际应用中，通常选择车道标志线上的特征点作为映射关系的已知点，通过手动标注或自动提取的方式获取其图像坐标和真实世界坐标，然后利用最小二乘法等优化算法求解单应性矩阵。

得到单应性矩阵后，即可将车辆检测和跟踪得到的图像坐标转换到路面坐标系中。设第t帧图像中车辆的像素坐标为(u_t,v_t)，则其对应的路面坐标(x_t,y_t)可按照下式计算：

$$\begin{bmatrix} wx_t \\ wy_t \\ w \end{bmatrix} = \boldsymbol{H} \begin{bmatrix} u_t \\ v_t \\ 1 \end{bmatrix} \tag{3.3}$$

$$x_t = \frac{wx_t}{w}, y_t = \frac{wy_t}{w} \tag{3.4}$$

其中，w为齐次坐标的尺度因子。

在获得车辆位置的路面坐标序列$\{(x_t, y_t)\}$后，即可计算车辆在相邻帧之间的位移，进而估计其速度。设相邻两帧的时间间隔为Δt，则第t帧到第$t+1$帧之间车辆的位移Δd为：

$$\Delta d = \sqrt{(x_{t+1} - x_t)^2 + (y_{t+1} - y_t)^2} \tag{3.5}$$

车辆在这一时间间隔内的平均速度v 可以估计为：

$$v = \frac{\Delta d}{\Delta t} \tag{3.6}$$

考虑到车辆运动的连续性和速度估计的稳定性，可以在多个连续帧上计算车辆的瞬时速度，然后取其平均值作为最终的速度估计结果。

综上所述，本设计提出的基于透视变换和坐标映射的车速估计方法，充分利用了视频图像中蕴含的时空信息，通过将车辆位置从图像坐标系映射到路面坐标系，实现了非接触式的车辆速度测量，为智能交通管理提供速度信息支撑。

3.3.3 车速检测界面设计

车速检测页面如图3.7所示。

车速检测界面用于显示车辆速度检测结果。提供“上传视频”和“使用摄像头”按钮，用于加载视频文件或启动实时摄像头进行车速检测。界面左侧显示原始视频帧，右侧显示带有速度标注的车辆检测结果视频帧。

下方设置播放控制按钮，包括“暂停/播放”和“重播”按钮，用于控制视频的暂停、播放和重新播放。提供“启动/停止检测”按钮，用于开始或停止车速检测过程。速度检测结果实时显示在视频帧上。

界面底部设置检测坐标、显示检测进度、处理模式选择（CPU或GPU）以及返回主界面按钮，用于返回系统主窗口。

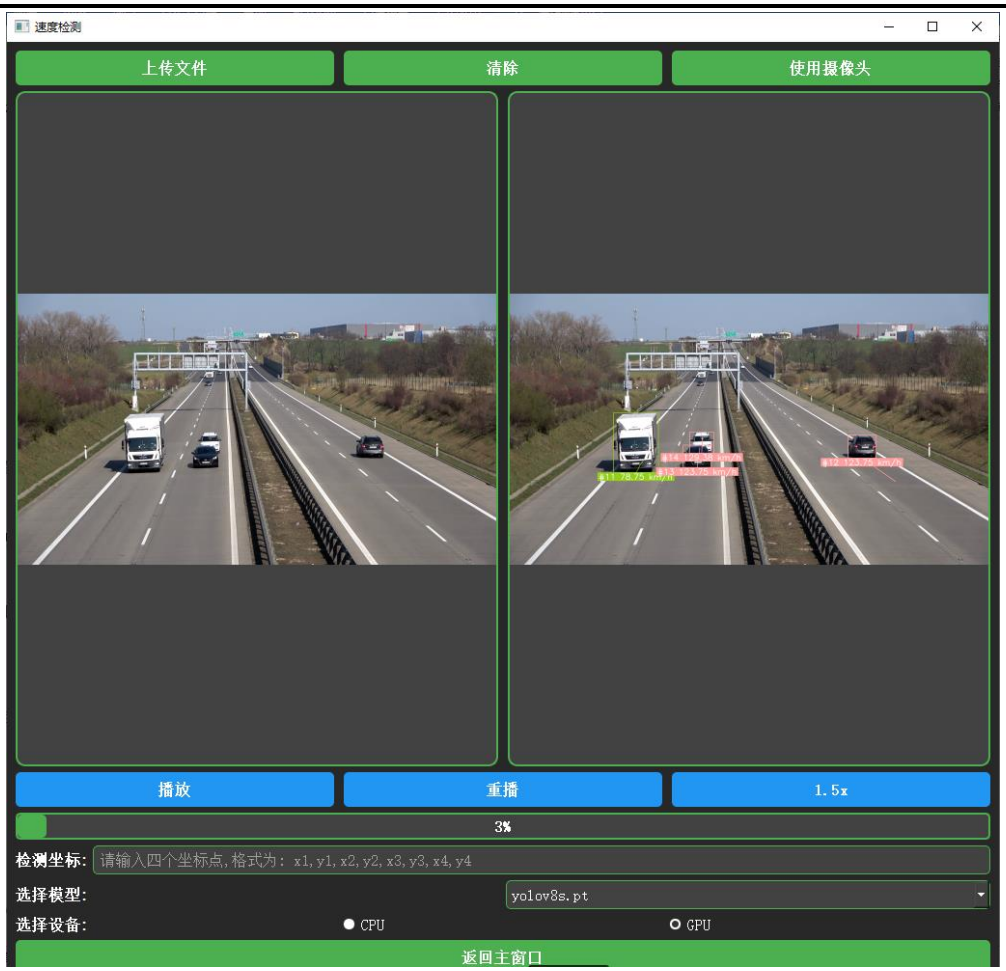

图3.7 车速检测界面

3.4 车流量统计模块

车流量统计是智能交通管理的重要功能之一，可以用于交通流量分析、拥堵预警等应用。本节将介绍车流量统计模块的设计与实现。

3.4.1 车流量统计流程

车流量统计的整体流程如图3.8所示。

具体步骤如下：

（1）车辆检测与跟踪：利用目标检测算法（YOLO）检测出视频帧中的车辆，并使用跟踪算法（ByteTrack）对车辆进行跨帧跟踪，得到车辆的运动轨迹。

（2）虚拟线圈设置：在视频画面中设置一条或多条虚拟线圈，用于判断车辆是否通过指定区域。

（3）车辆计数：根据车辆轨迹与虚拟线圈的位置关系，判断车辆是否通过虚拟线圈，并对通过的车辆进行计数。

（4）车流量统计：对一定时间间隔内通过虚拟线圈的车辆数量进行统计，得到

车流量数据。车流量按车道、方向等维度进行细分统计。

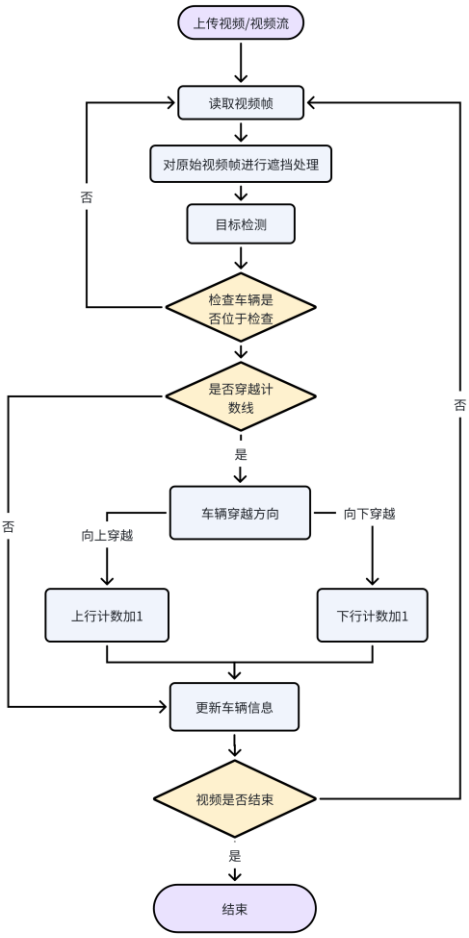

图3.8 车流量统计流程图

3.4.2 车辆计数方法

车辆计数是车流量统计的核心部分，本设计设计了一种基于车辆轨迹与计数线交互的计数方法。通过判断车辆的运动轨迹是否穿过预定义的计数线，实现车辆的计数和方向分类。

首先，在视频画面中预定义一条或多条计数线，用于判断车辆是否穿过。计数线可以表示为一个线段，用起点坐标(x_1, y_1)和终点坐标(x_2, y_2)来定义：

$$line = [(x_1, y_1), (x_2, y_2)] \tag{3.7}$$

其中，x_1, y_1, x_2, y_2分别表示计数线起点和终点的像素坐标。

然后，对检测到的车辆进行跟踪，形成车辆的运动轨迹。车辆轨迹可以表示为一系列的边界框坐标序列：

$$trajectory = [(x_1, y_1, w_1, h_1), (x_2, y_2, w_2, h_2), \dots, (x_n, y_n, w_n, h_n)] \tag{3.8}$$

其中，(x_i, y_i, w_i, h_i)表示第i帧中车辆边界框的中心坐标、宽度和高度。

接下来，判断车辆的运动轨迹是否穿过了计数线。对于每一个被跟踪的车辆，获取其当前位置(x_c, y_c)和上一帧位置(x_p, y_p)，计算当前位置和上一帧位置与计数线的交点坐标(x_i, y_i)：

$$\begin{cases} x_i = \dfrac{(y_2 - y_1)(x_c - x_p) - (x_2 - x_1)(y_c - y_p)}{(y_2 - y_1)(x_p - x_1) - (x_2 - x_1)(y_p - y_1)} \\ y_i = \dfrac{(x_2 - x_1)(y_p - y_1) - (y_2 - y_1)(x_p - x_1)}{(y_2 - y_1)(x_p - x_1) - (x_2 - x_1)(y_p - y_1)} \end{cases} \tag{3.9}$$

判断交点(x_i, y_i)是否在计数线上，需要满足以下条件：

交点的x坐标在计数线的x坐标范围内，即：

$$\min(x_1, x_2) \le x_i \le \max(x_1, x_2) \tag{3.10}$$

交点的y坐标在计数线的y坐标范围内，即：

$$\min(y_1, y_2) \le y_i \le \max(y_1, y_2) \tag{3.11}$$

如果交点在计数线上，则认为车辆穿过了计数线。根据车辆轨迹与计数线的相对位置关系，判断车辆穿过计数线的方向：

如果(x_p, y_p)在计数线的左侧，(x_c, y_c)在计数线的右侧，则车辆为上行方向。

如果(x_p, y_p)在计数线的右侧，(x_c, y_c)在计数线的左侧，则车辆为下行方向。

根据车辆穿过计数线的情况，对车辆进行计数。每当有车辆穿过计数线时，根据其穿过方向，将计数结果累加到对应的方向计数器中：

如果车辆为上行方向，则上行计数器加1：

$$count_{up} = count_{up} + 1 \tag{3.12}$$

如果车辆为下行方向，则下行计数器加1：

$$count_{down} = count_{down} + 1 \tag{3.13}$$

为了避免重复计数，需要对每辆车维护一个计数标志位。当车辆第一次穿过计数线时，将其计数标志位设为True，表示已经计数过。在后续帧中，如果车辆已经计数过，则不再重复计数。

通过以上计算方法，实现了一个基于车辆轨迹与计数线交互的车辆计数算法。该算法结合了目标检测、跟踪和几何计算，能够有效地统计道路上的车流量，为交通管理和决策提供数据支持。

3.4.3 车流量统计界面设计

车流量统计页面如图3.9所示。

车流量统计界面用于显示车流量统计结果。提供“上传视频”和“启动摄像头”按钮，用于加载视频文件或启动实时摄像头进行车流量统计。界面左侧显示原始视频画面，右侧显示带有车辆计数标注的视频画面。

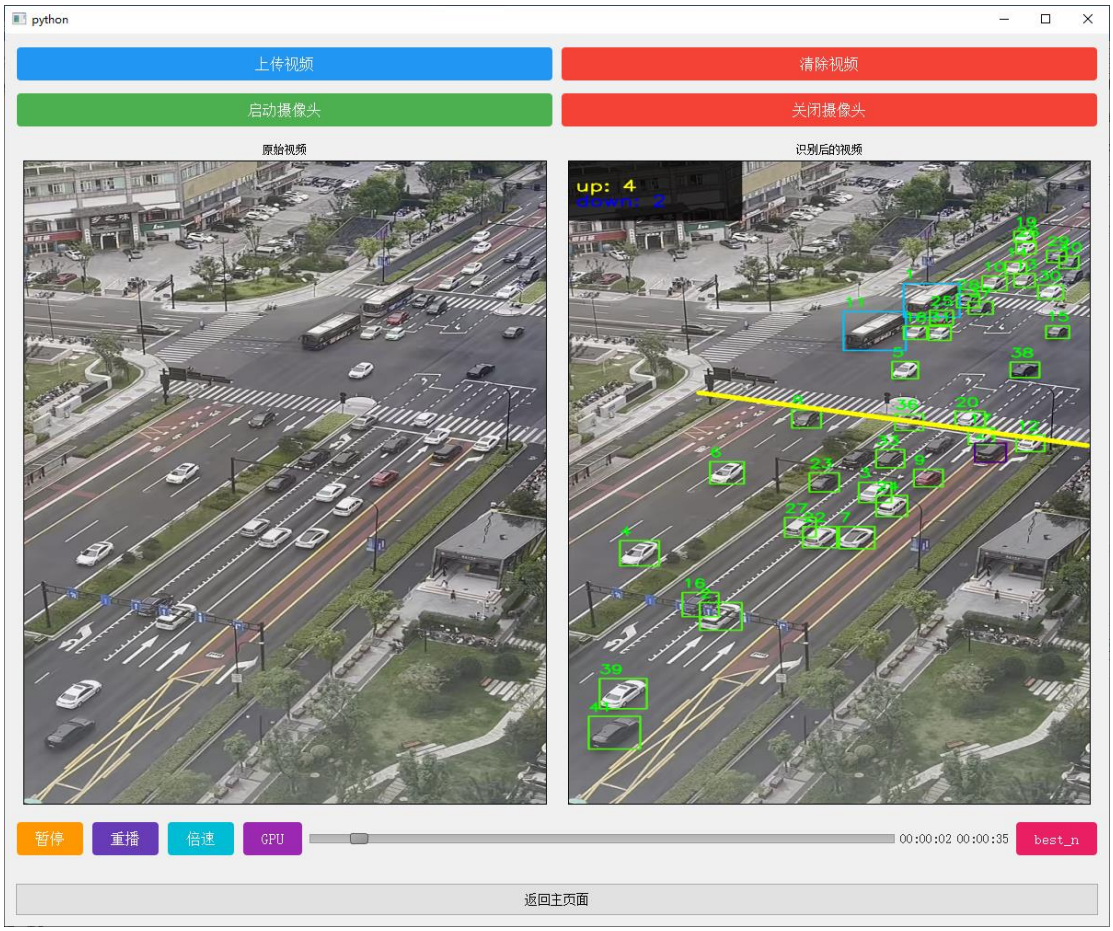

图3.9 车流量统计界面

在视频画面上，使用绿色矩形框标注检测到的车辆，并在矩形框旁边显示车辆计数结果。在视频画面的左上角叠加实时的车流量统计信息。

下方设置播放控制按钮，包括播放、暂停、倍速播放和进度条，用于控制视频的播放、暂停、调整播放速度和跳转到指定位置。提供GPU切换按钮和模型切换按钮，用于车流量统计处理。界面底部显示统计进度条，实时更新车流量统计的进度。右下角设有“返回主界面”按钮，用于返回系统主界面。

3.5 车辆停留时间检测模块

车辆停留时间检测是智能车辆管理的一项重要功能，可以用于识别异常停车、交通拥堵等情况，为交通管控和疏导提供决策依据。本节将介绍车辆停留时间模块的设计与实现。

3.5.1 停留时间检测流程

车辆停留时间检测的整体流程如图3.10示。

具体步骤如下：

（1）视频输入：从视频文件或摄像头获取视频源。

（2）帧读取：逐帧读取视频中的图像帧。

（3）目标检测：使用目标检测模型对每一帧图像进行目标检测，识别出图像中的车辆。

（4）目标跟踪：使用目标跟踪算法（ByteTrack）对检测到的车辆进行跟踪，生成每个车辆的唯一ID和运动轨迹。

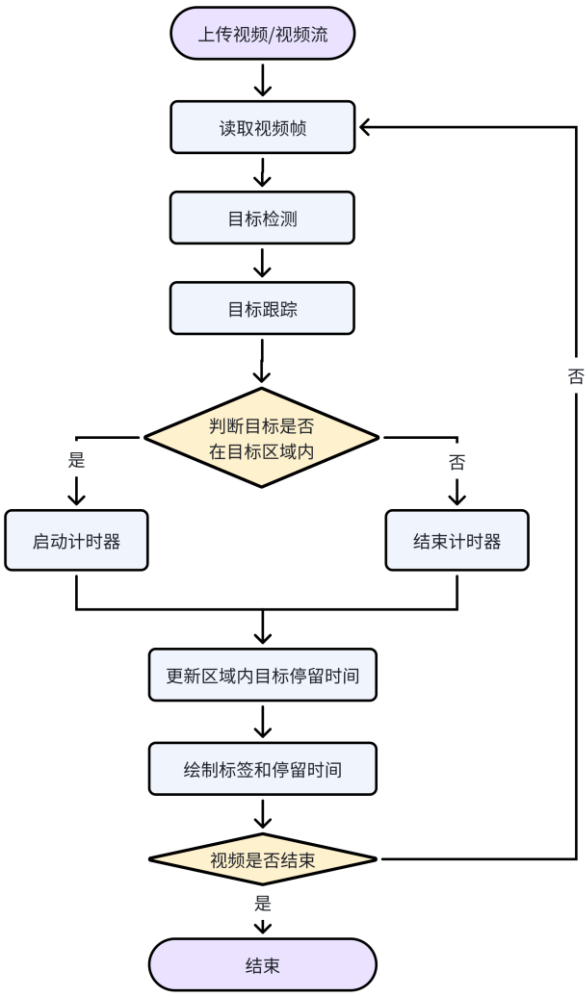

图3.10 车辆停留时间检测流程图

（5）目标区域（ROI）定义：根据预定义的JSON配置文件，确定视频中的目标区域，通常是车辆停留的区域。

（6）停留时间计算：对于每一帧，检查跟踪到的车辆是否在目标区域内。如果车辆在区域内，启动计时器计算其停留时间；如果车辆离开区域，停止计时器。

（7）结果可视化：在每一帧图像上绘制检测到的车辆边界框、跟踪ID以及车辆在目标区域内的停留时间。

（8）视频输出：将处理后的视频帧连续输出，形成处理后的视频。

（9）用户交互：提供用户界面，允许用户控制视频的播放、暂停、调整播放速度等，以及显示视频处理进度。

3.5.2 停留时间计算方法

车辆停留时间检测是智能交通系统的重要功能之一，通过分析车辆在特定区域内的停留时间，可以了解道路的通行状况，发现异常停车行为，为交通管理提供数据支持。本设计设计了一种基于目标区域（Region of Interest，ROI）和车辆跟踪的停留时间计算方法。

首先，在视频画面中定义一个或多个ROI，表示需要进行停留时间分析的区域。ROI可以是任意形状的多边形，用一系列顶点坐标来描述。ROI由n个顶点构成，其可以表示为：

$$ROI = [(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)] \tag{3.14}$$

其中，(x_i, y_i)表示第i个顶点的像素坐标。

接下来，对视频帧中检测到的车辆进行跟踪，获取每辆车在连续帧中的位置和运动轨迹。

为了判断车辆是否位于ROI内部，使用射线法。射线法的基本原理是：从车辆中心点(c_x, c_y)向右引出一条射线，统计射线与ROI边界的交点数[20]。如果交点数为奇数，则车辆位于ROI内部；如果交点数为偶数，则车辆位于ROI外部。

当车辆进入ROI时，记录下进入时刻t_{in}，并初始化停留时间Δt为0。在后续帧中，如果车辆仍然位于ROI内，计算当前时刻t与进入时刻t_{in}的时间差，并将其累加到停留时间Δt中。如果车辆离开了ROI，则停止更新其停留时间，并记录下累计的停留时长。

综上所述，本设计的车辆停留时间计算方法，通过定义目标区域，跟踪车辆轨迹，判断车辆与ROI的位置关系，记录进出ROI的时间，计算停留时间。该方法充分利用了计算机视觉和车辆跟踪技术，具有一定的实用性和扩展性，为智能交通系统的开发和应用提供了新的思路。

3.5.3 车辆停留时间分析界面设计

车辆停留时间页面如图3.11所示。

图3.11 车辆停留时间分析界面

车辆停留时间分析界面用于显示车辆在特定区域的停留时间分析结果。界面提供“上传视频”按钮，用于上传分析的视频文件。“上传检测区域文件”按钮用于上传JSON配置文件，可设置分析检测区域参数。“清除视频”按钮用于清除已上传的视频。

界面左侧显示原始视频画面，右侧显示带有停留时间标注的分析结果视频画面。在分析结果视频帧上，使用不同颜色的矩形框标注不同停留时间阈值下的车辆，并在矩形框旁边标注车辆ID和具体的停留时间。

下方设置播放控制按钮，包括播放、暂停按钮和进度条，用于控制视频的播放、暂停和倍速。界面右下角设有返回按钮，用于返回上一级界面。

4 边缘设备移植与优化加速

为了进一步提高车辆停留时间检测模块的执行效率并支持在边缘设备上部署，

本设计还提供了该模块的C++实现版本。通过充分利用C++的高性能特性和针对边缘设备的优化策略，C++版本的停留时间检测模块在边缘设备上能够稳定运行，处理效率较Python版本有显著提升。这为智能车辆识别系统在资源受限的边缘设备上的实际部署提供了一种可行的解决方案，极大地扩展了系统的应用场景和实用价值。

4.1 C++代码移植

本节将详细介绍将智能车辆识别系统中的车辆停留时间检测模块从Python移植到C++的过程。在移植过程中，着重考虑了边缘设备的资源限制和实时性要求，对原有的算法和数据结构进行了优化和改进。同时，为了保持代码的可读性和可维护性，在C++版本的实现中遵循了模块化的设计原则，并提供了清晰的接口定义。

4.1.1 C++版本的系统架构

C++版本的智能车辆识别系统在整体架构上与Python版本保持了高度的一致性，但在模块的划分和接口的定义上进行了进一步的优化。系统主要由以下几个模块组成：

（1）视频输入与预处理模块：负责从视频文件读取输入视频流，并对每一帧图像进行必要的预处理操作，如尺寸调整、颜色空间转换等。该模块利用OpenCV库提供的相关功能实现了高效的视频读取和解码。

（2）YOLOv8目标检测模块：使用经过量化和优化的YOLOv8模型对输入的视频帧进行车辆目标检测[21]。为了在边缘设备上获得实时的检测性能，采用了基于NCNN框架的YOLOv8模型实现。该模块的输出为每一帧图像中检测到的车辆目标的边界框和类别信息。

（3）检测结果后处理模块：对YOLOv8的检测结果进行进一步的后处理，包括根据置信度阈值过滤低质量的检测框，对重叠度较高的检测框进行非极大值抑制等。该模块使用了自定义的数据结构和算法，提高了后处理的效率。

（4）ByteTrack多目标跟踪模块：在检测结果的基础上，利用ByteTrack算法实现车辆目标的多目标跟踪。通过将当前帧的检测结果与前一帧的跟踪结果进行关联，生成每个车辆目标的运动轨迹。该模块采用了优化后的卡尔曼滤波器和匈牙利算法，提升了跟踪的实时性和鲁棒性。

（5）车辆停留时间分析模块：基于跟踪得到的车辆轨迹，对预定义的目标区域进行停留时间分析。通过判断车辆中心点是否位于目标区域内，并结合视频帧率信息，计算每个车辆在各个区域内的累计停留时间。该模块利用了高效的几何算法实

现区域判断，并设计了时间阈值和空间阈值等参数以提高分析的准确性。

（6）可视化与输出模块：根据车辆检测、跟踪和停留时间分析的结果，在视频帧上绘制相应的标注信息，如车辆边界框、跟踪轨迹、停留时间等。该模块使用OpenCV的绘图功能实现了实时的可视化效果。处理后的视频帧可以实时显示在屏幕上，也可以选择保存到输出视频文件中。

以上模块通过明确的接口定义和数据结构进行解耦，每个模块负责特定的功能，可以独立地进行开发和测试。这种模块化的架构设计不仅提高了系统的可维护性，也为后续的功能扩展和算法优化提供了便利。在实现过程中，还充分利用了C++的语言特性，使得代码更加简洁、安全和高效。

4.1.2 关键模块的C++实现

在C++版本的智能车辆识别系统中，有几个关键模块的实现对系统的整体性能和功能起到了决定性的作用。下面重点介绍其中三个核心模块：车辆停留时间分析模块、ByteTrack多目标跟踪模块和可视化与输出模块。

（1）车辆停留时间分析模块

车辆停留时间分析是整个系统的核心功能，其目标是统计每辆车在预定义的目标区域内的累计停留时间。为了实现高效、准确的停留时间分析，在C++版本中设计了两个关键的类：PolygonZone和FPSBasedTimer。

PolygonZone类表示一个多边形形状的目标区域，其构造函数接受一个Point类型的向量，表示多边形的顶点坐标。该类的核心功能是判断给定的点是否位于多边形区域内部。采用了计算几何中的射线法来实现该功能：从待判断点向任意方向引出一条射线，统计射线与多边形边界的交点数。交点数为奇数则点在多边形内部，否则在外部。射线法的时间复杂度为O(n)，其中n为多边形的顶点数，满足实时分析的需求。

FPSBasedTimer类则用于根据视频帧率信息计算车辆在目标区域内的停留时间。该类的构造函数接受视频的帧率作为参数，内部维护了一个以车辆ID为索引的哈希表，记录每辆车的初次出现帧号。当车辆进入目标区域时，记录其进入帧号；当车辆离开区域时，根据当前帧号和进入帧号，计算停留时间并累加到相应的计时器中。为了处理车辆短暂离开区域又快速返回的情况，引入了时间阈值参数，只有车辆离开区域的连续帧数超过该阈值，才认为其真正离开，避免了重复计时。

在系统的主处理循环中，将每一帧的检测结果和跟踪结果输入到车辆停留时间

分析模块。对于每个目标区域，遍历所有跟踪到的车辆，判断其中心点是否位于区域内部，并相应地启动或更新计时器。在绘制停留时间标签时，将车辆ID与格式化后的停留时间字符串拼接，以“ID：时：分：秒”的形式显示在相应的车辆边界框上方。

通过PolygonZone和FPSBasedTimer的配合，车辆停留时间分析模块以每秒超过30帧的速度实时分析每辆车在各个区域的停留情况，为交通管理人员提供了直观、准确的决策依据。

（2）ByteTrack多目标跟踪模块

ByteTrack是一种简洁、高效的多目标跟踪算法，在C++版本的系统中对其进行了实现和优化。ByteTrack的核心思想是将检测器得到的目标检测结果与已有的跟踪轨迹进行关联，生成每个目标的运动轨迹，并对新出现的目标分配唯一的跟踪ID。

实现中，ByteTrack类的主要功能是接受当前帧的检测结果作为输入，内部维护了三个关键的数据结构：aktive_tracks、lost_tracks和removed_tracks，分别表示当前激活的跟踪轨迹、丢失的跟踪轨迹和需要删除的跟踪轨迹。

跟踪过程分为以下几个步骤：首先，对于每个激活的跟踪轨迹，预测当前时刻的位置；然后，将预测位置与当前帧检测结果进行匹配，找到IoU最大的检测框作为匹配结果；对于未匹配上的跟踪轨迹，将其标记为丢失状态，并记录丢失的时间；对于连续丢失时间超过一定阈值的跟踪轨迹，将其从aktive_tracks中删除，并加入到removed_tracks；最后，对于未匹配上任何跟踪轨迹的检测结果，为其创建新的跟踪轨迹，分配唯一的跟踪ID，并加入到aktive_tracks。

在匹配检测结果与跟踪轨迹时，采用了改进的匈牙利算法。首先计算每个检测结果与每个跟踪轨迹的IoU值，形成一个代价矩阵；然后，利用该矩阵进行最优匹配，找到总体代价最小的匹配方案。匈牙利算法的时间复杂度为O(n^3)，其中n为检测结果和跟踪轨迹的总数。考虑到在实际场景中，每一帧的目标数量通常不会很大，该算法可以满足实时跟踪的需求。

为了进一步提高跟踪的鲁棒性，在ByteTrack中引入了卡尔曼滤波器，用于预测和更新每个跟踪轨迹的状态。卡尔曼滤波器是一种递归的贝叶斯估计方法，通过融合观测值和预测值，对目标状态进行最优估计。在的实现中，状态向量包括目标的位置和速度信息，观测值为匹配到的检测框的位置。通过卡尔曼滤波，可以得到更加平滑、准确的跟踪轨迹，有效地应对目标的遮挡、丢失等问题。

ByteTrack类的输出为一个TrackingDetection类型的向量，每个元素包含了跟踪目标的边界框、跟踪ID、类别等信息。这些信息将被传递给车辆停留时间分析模块和可视化模块，用于后续的处理和显示。

（3）可视化与输出模块

为了直观地展示车辆检测、跟踪和停留时间分析的结果，设计了可视化与输出模块，利用OpenCV库提供的绘图功能，在每一帧图像上叠加相关的标注信息。

在绘制检测框时，使用OpenCV的绘图函数绘制矩形，颜色根据车辆类别的不同而有所区分。在矩形框的上方，绘制车辆类别和置信度信息。考虑到在复杂场景下可能出现大量的检测框，为了避免标注信息的重叠遮挡，对标注文字的背景区域进行了透明度处理，提高了可读性。

对于跟踪轨迹的绘制，将每辆车的历史位置存储在一个固定长度的缓冲区中，并使用OpenCV的绘图函数将这些位置点连接成线，形成运动轨迹。通过设置不同的颜色和透明度，可以直观地展示出车辆的运动方向和速度。在轨迹的起点，绘制车辆的跟踪ID，方便识别和统计。

停留时间信息的可视化是本模块的一大亮点。对于每个目标区域，使用OpenCV的绘图函数绘制多边形区域，并设置半透明的填充颜色。在区域内停留的车辆上方，绘制停留时间标签，格式为“ID：时：分：秒”。通过合理设置字体大小、颜色和背景，标签信息清晰、醒目，即使在车辆密集的场景下也能够准确无误地显示。

考虑到实际应用中可能需要保存处理后的视频，还提供了视频输出的功能。使用OpenCV的VideoWriter类，可以方便地将处理后的视频帧写入到指定的输出文件中。输出视频的编码格式、帧率等参数可以根据需求进行配置，确保视频的质量和兼容性。

总的来说，可视化与输出模块充分利用了OpenCV库的图像绘制和视频处理功能，设计了直观、美观的可视化效果，使得车辆识别系统的结果易于理解和分析。通过实时绘制检测框、跟踪轨迹和停留时间标签，可以及时掌握道路上车辆的动态。

为了提高车辆停留时间检测模块的执行效率并支持在边缘设备上部署，还提供了该模块的C++实现版本。C++版本的停留时间检测模块在边缘设备上能够稳定运行，处理效率较Python版本有显著提升。这为智能车辆识别系统在边缘设备上的部署提供了一种可行的解决方案，扩展了系统的应用场景。

4.2 优化加速视频FPS

本节将介绍在边缘设备上对YOLOv8模型进行优化和加速的方法，包括模型量化、线程池优化等。通过这些优化手段，将提升YOLOv8在边缘设备上的推理速度，达到实时视频检测的要求。

4.2.1 模型导出与量化

为了在边缘设备上部署YOLOv8模型，需要将PyTorch格式的模型转换为RKNN格式[22]。首先，使用YOLOv8官方提供的export_onnx.py脚本将模型导出为ONNX格式。在导出过程中，对模型进行了一些改动，将其移到CPU上执行，以提高NPU的推理效率。

接下来，使用RKNN-Toolkit将ONNX模型转换为RKNN格式[23]。在转换过程中，对模型进行了量化操作。量化是一种常用的模型压缩和加速技术，通过将模型权重和激活值从浮点数转换为定点数，可以减小模型体积，降低计算复杂度，从而加快推理速度。使用RKNN-Toolkit提供的量化工具，生成了int8位定点数的量化RKNN模型。

4.2.2 线程池优化

为了充分利用边缘设备的计算资源，引入了线程池机制对推理过程进行优化。传统的单线程模式无法发挥NPU的并行计算能力，而简单的多线程模式又会引入线程创建和销毁的开销。线程池维护了一组预创建的线程，可以重复利用，避免了频繁创建和销毁线程的开销。

通过设计一个线程池管理器，用于创建和管理线程池。线程池中的线程数可以根据需求进行配置。每个线程负责读取视频帧，进行预处理，然后将任务提交到线程池的任务队列中。线程池中的工作线程从任务队列中获取任务，调用RKNN Runtime执行推理，并进行后处理。通过这种方式，多个线程可以并行执行推理任务，充分利用NPU的并行计算能力。

4.3 移植效果性能评估

为了全面评估优化后的YOLOv8模型在边缘设备上的性能表现，从处理效率和资源占用两个方面进行了测试和分析。在处理效率方面，使用了FPS（Frames Per Second）作为衡量指标。FPS表示每秒处理的图像或视频帧数，反映了模型的推理速度。FPS越高，意味着模型可以在更短的时间内处理更多的数据，实时性能越好。

在资源占用方面，重点关注了NPU（Neural Processing Unit）的利用率。NPU是

专门用于加速深度学习运算的处理器，与CPU和GPU相比，它能够以更高的能效比、更低的功耗、更小的芯片面积来处理AI任务。因此，NPU特别适合于移动端和边缘设备上的AI应用。通过监测NPU的利用率，可以了解模型在运行时对计算资源的占用情况。

综合FPS和NPU利用率这两个指标，可以全面地评估YOLOv8模型在边缘设备上的表现。FPS的高低反映了模型的处理效率，而NPU利用率则体现了模型对芯片资源的使用情况。二者结合，可以更好找到性能和功耗之间的最佳平衡点。

（1）浮点模型和量化模型评估测试

为了评估量化对模型性能的影响，分别测试了浮点模型和量化模型在边缘设备上的推理速度。结果如表4.1和表4.2所示，量化模型的推理速度明显高于浮点模型，帧率从4FPS提升到9FPS，速度提升将近一倍。同时，监控了边缘设备的NPU利用率，发现量化模型的NPU利用率略有降低，有优化空间。

表4.1 浮点模型FPS和NPU利用率（%）

FPS	Core0	Core1	Core2
4.826255	30	0	0
4.405286	31	0	0
4.369513	29	0	0
4.215369	30	0	0

表4.2 量化模型FPS和NPU利用率（%）

FPS	Core0	Core1	Core2
9.625423	20	0	0
9.523365	21	0	0
9.311611	22	0	0
9.526121	20	0	0

（2）不同线程数评估测试

通过测试了不同线程数对推理速度的影响。结果表明，随着线程数的增加，推理速度不断提高。当线程数达到15时，浮点模型的推理速度从5FPS提升到29FPS，如表4.3所示。量化模型的推理速度从9FPS提升到80FPS，如表4.4所示。同时，监控了边缘设备的NPU利用率，发现线程数增加时，NPU利用率显著提高，多个NPU核心

被充分利用。

表4.3 浮点模型15个线程时FPS和NPU利用率（%）

FPS	Core0	Core1	Core2
16.226136	90	89	91
20.166612	92	93	95
25.315626	93	92	93
29.325616	94	95	93

表4.4 量化模型15个线程时FPS和NPU利用率（%）

FPS	Core0	Core1	Core2
45.512512	87	76	69
60.262163	86	73	70
76.261611	88	76	73
80.256192	84	75	75

综合以上评估结果，可以得出结论：通过模型量化、线程池优化等方法，成功地将YOLOv8移植到边缘设备上，并取得了明显的性能提升。优化后的模型在保证检测精度的同时，实现了实时视频检测的目标。系统资源占用合理，稳定性和可靠性良好，满足实际应用的需求。这为进一步在边缘设备上部署先进的目标检测算法提供了有益的探索和参考。

5 总结与展望

5.1 工作总结

本系统针对智能车辆识别这一领域，设计并实现了一套多功能的智能识别系统。该系统基于深度学习技术，实现了车辆属性识别、车速检测、车流量统计和停留时间检测等关键功能，为智能车辆管理提供了全面、精准的数据支持。

在车辆属性识别方面，本系统采用CNN实现了车牌识别和车身颜色识别。针对车牌识别，设计了一种端到端的识别框架，避免了字符分割导致的误差累积。针对车身颜色识别，提出了一种融合颜色直方图特征和分类器的识别方法，在准确性和实时性方面取得了很好的平衡。

在车速检测方面，本系统创新性地引入了透视变换和目标跟踪相结合的方法。通过将成像平面的车辆映射到鸟瞰视角，并跟踪其位移变化，实现了非接触式的视

频车速估计，为传统车速检测方法提供了一种灵活、经济的替代方案。

针对车流量统计和停留时间检测，本系统巧妙地设计了虚拟线圈和目标区域分析的方法。利用视频中车辆轨迹与线圈、区域的关系分析，高效而准确地实现了车流统计与停留时间分析。这对拥堵识别、交通优化等具有重要的应用价值。

在工程实现方面，本系统还致力于将算法移植到边缘边缘设备平台，并进行了深入的速度优化。通过模型量化、并行化等手段，在保证精度的同时，将帧率从最初的5FPS提升到了80FPS，基本可满足实时视频分析的需求。

总的来说，本系统围绕智能车辆识别这一主题，从算法到工程实现进行了全面、深入的研究与实践，成功开发了一个集多功能于一体、性能优异的智能车辆识别系统。

5.2 未来展望

基于本系统的研究基础，未来还可以在以下几个方面继续深化和拓展：

（1）融合异构数据源。除视频数据外，车辆感知还可以融合雷达、车联网等数据，形成更鲁棒的多源异构数据融合方案。

（2）轻量化识别模型。为了进一步提高模型的实时性和可移植性，可以开发一些轻量化的识别模型，在边缘计算设备上也能高效运行。

（3）开发自适应学习方法。面对复杂多变的交通场景，需开发一些自适应学习的方法，使系统能够自动适应环境变化，不断进化。

（4）拓展到行人和非机动车。将智能识别的对象从机动车拓展到行人和非机动车，全面感知道路交通的状态。

（5）与信号灯、诱导屏等交通设施联动。将车辆的识别结果与交通基础设施结合，形成车路协同的闭环控制，实现更智能的交通管理。

（6）建立车辆诊断与预警模型。基于车辆的识别数据，建立车辆的故障诊断与风险预警模型，为车辆安全提供预防性的支持。

智慧交通的发展离不开车辆精准感知能力的提升。相信随着人工智能等技术的不断进步，智能车辆识别系统必将从感知向理解、决策的方向深化，在智慧城市中发挥越来越重要的作用。本系统的研究成果将为后续的深入研究奠定良好的基础。

致谢

在完成这篇论文的过程中，我要向我的指导老师王国政老师表达最真挚的感谢。从论文主题的选定到相关文献资料的选择和收集，再到论文的修改完善，王国政老师在每一个环节都倾注了大量的心血和精力。王国政老师总是不厌其烦地解答我的疑惑，给予我细致入微的指导和帮助。王国政老师认真负责的态度和严谨的治学理念深深地影响了我，这不仅让我在学术研究上受益匪浅，更将成为我今后学习和工作的宝贵财富。

此外，我还要感谢学院里所有教导过我的老师。是他们的谆谆教诲，让我的知识得以积累；是他们的言传身教，让我的人格得以塑造。每位老师独特的教学风格和渊博的学识，都给我留下了深刻的印象。从他们身上，我学到了如何思考问题、如何解决问题，更学到了做人做事的道理。这些都是我人生中最宝贵的财富，在此，我再次向所有老师表示由衷的感谢。

参考文献

[1] Li, Hui, Peng Wang, and Chunhua Shen. "Toward end-to-end car license plate detection and recognition with deep neural networks." IEEE Transactions on Intelligent Transportation Systems 20.3 (2018): 1126-1136.

[2] Shvai, Nadiya, Abul Hasnat, and Amir Nakib. "Multiple auxiliary classifiers GAN for controllable image generation: Application to license plate recognition." IET Intelligent Transport Systems 17.1 (2023): 243-254.

[3] Wang, Zhangjing, Yu Wu, and Qingqing Niu. "Multi-sensor fusion in automated driving: A survey." Ieee Access 8 (2019): 2847-2868.

[4] 张健. 基于注意力机制的车辆检测算法研究[D].大连海事大学,2022.DOI:10.26989/d.cnki.gdlhu.2022.000649.

[5] 于淏. 复杂场景下的车牌识别算法研究[D].杭州电子科技大学,2024.DOI:10.27075/d.cnki.ghzdc.2023.000411.

[6] 曾伟良,聂佩林,何兆成,等.结合神经网络和数据融合技术的城市路段速度估计[J].计算机应用研究,2011,28(12):4459-4462.

[7] 范靖,山世玉,林先山.智能驾驶汽车机器视觉关键技术分析[J].汽车实用技术,2023,48(21):173-178.DOI:10.16638/j.cnki.1671-7988.2023.021.035.

[8] 盛卓. 基于视频图像的车型识别技术的研究与实现[D].哈尔滨工程大学,2013.

[9] 李绍骞,程鑫,周经美,等.基于车辆外观特征和帧间光流的目标跟踪算法[J].应用科学学报,2024,42(01):103-118.

[10] 梁智博. 无人机目标跟踪和识别算法研究[D].哈尔滨工业大学,2022.DOI:10.27061/d.cnki.ghgdu.2022.002635.

[11] 刘晗,张醒敏,周炳清,等.京哈北高速公路隧道AI事件监测系统探究[J].中国公路,2023(19):94-97.DOI:10.13468/j.cnki.chw.2023.19.016.

[12] 王源顺. 基于深度学习的图像描述生成算法的研究及应用[D].贵州大学,2022.DOI:10.27047/d.cnki.ggudu.2021.003076.

[13] Lee, Hee Seok, and Kang Kim. "Simultaneous traffic sign detection and boundary estimation using convolutional neural network." IEEE Transactions on Intelligent Transportation Systems 19.5 (2018): 1652-1663.

[14] Alsanad, Hamid R., et al. "YOLO-V3 based real-time drone detection

algorithm." Multimedia tools and applications 81.18 (2022): 26185-26198.

[15] Cao, Jie, et al. "A long-memory pedestrian target tracking algorithm incorporating spatiotemporal trajectory feature enhancement model." Digital Signal Processing 135 (2023): 103929.

[16] Gunjal, Pramod R., et al. "Moving object tracking using kalman filter." 2018 International Conference On Advances in Communication and Computing Technology (ICACCT). IEEE, 2018.

[17] Wang, Kun, and Maozhen Liu. "YOLOv3-MT: A YOLOv3 using multi-target tracking for vehicle visual detection." Applied Intelligence 52.2 (2022): 2070-2091.

[18] Chen, Pan, Xiang Bai, and Wenyu Liu. "Vehicle color recognition on urban road by feature context." IEEE Transactions on Intelligent Transportation Systems 15.5 (2014): 2340-2346.

[19] 朱冰冰. 基于深度学习的3D车道线检测方法研究[D].天津职业技术师范大学,2023.DOI:10.27711/d.cnki.gtjgc.2023.000001.

[20] 谢少菲,高铁军,魏恋欢.基于网格的多点在多边形内高效判定方法[J].计算机工程与设计,2024,45(01):139-145.DOI:10.16208/j.issn1000-7024.2024.01.018.

[21] 张金雷,杨健,刘晓冰,等.基于计算机视觉的轨道交通站内火灾检测与定位[J/OL].交通运输系统工程与信息:1-14[2024-05-03].http://kns.cnki.net/kcms/detail/11.4520.u.20240422.1755.002.html.

[22] 古宇轩. 基于RK3399Pro的人员检测和计数系统的研究与实现[D].南京邮电大学,2023.DOI:10.27251/d.cnki.gnjdc.2023.000207.

[23] Ismagilov, Radik. "Performance Evaluation of the Rockchip Systems-on-Chip Through YOLOv4 Object Detection Model." 2023 IEEE Ural-Siberian Conference on Biomedical Engineering, Radioelectronics and Information Technology (USBEREIT). IEEE, 2023.